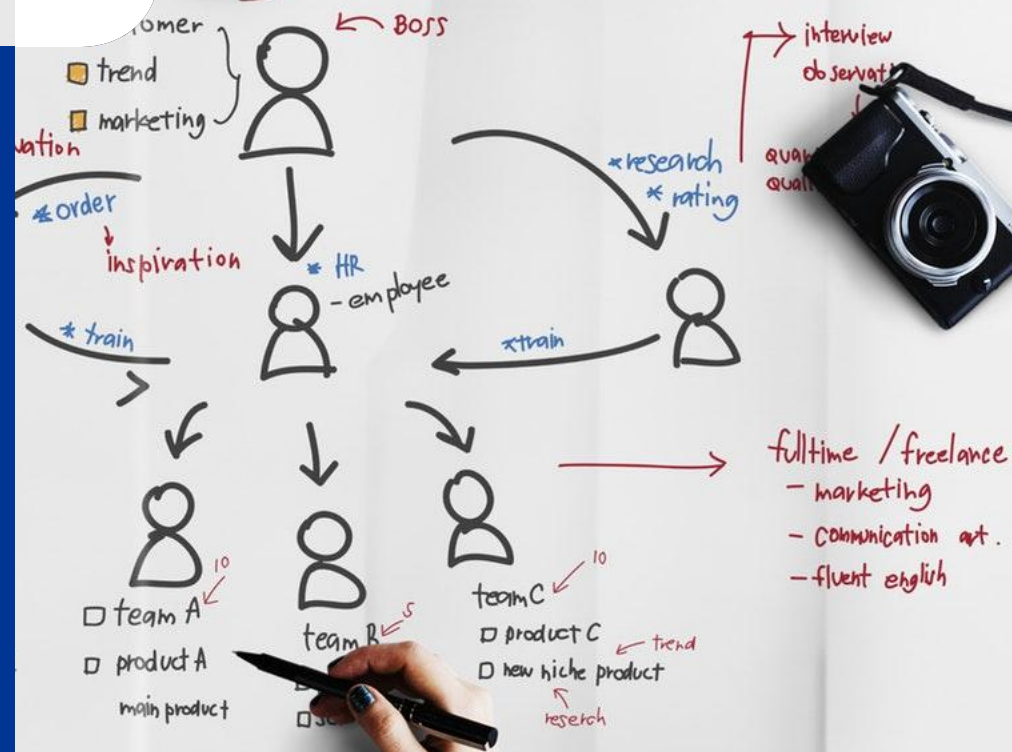


Σdavante

La Liga Fantasy Generator

Félix Caballero Peña
Gabriel Sánchez Heredia



Control de
versiones

GitHub



Pruebas unitarias

JUnit



Pruebas unitarias JUnit5



The screenshot shows an IDE with a project named 'LaLiga-Fantasy-Generator-1'. The left sidebar displays the project structure, including a 'Test' directory containing 'testUnitariosProyecto'. The main editor window shows the code for 'testUnitariosProyecto.java'.

```
1 import clasesProyecto.Equipo;
2 import org.junit.jupiter.api.BeforeEach;
3 import org.junit.jupiter.api.Test;
4
5 import java.time.Year;
6
7 import static org.junit.jupiter.api.Assertions.*;
8
9 public class testUnitariosProyecto {
10
11     private Equipo equipo1;
12     private Equipo equipo2;
13
14     @BeforeEach
15     public void setUp() {
16         equipo1 = new Equipo(nombre: "FC Barcelona", valoracion: 5, puntos: 0, PJ: 0, PG: 0, PE: 0, PP: 0, GF: 0, GC: 0, DG: 0);
17         equipo2 = new Equipo(nombre: "Real Betis", valoracion: 4.5f, puntos: 0, PJ: 0, PG: 0, PE: 0, PP: 0, GF: 0, GC: 0, DG: 0);
18     }
19
20     @Test
21     public void testGetNombre() {
22         assertEquals("FC Barcelona", equipo1.getNombre());
23     }
24
25     @Test
26     public void testSumarPuntos() {
27         equipo2.sumarPuntos(1);
28         equipo2.sumarPuntos(3);
29         equipo2.sumarPuntos(3);
30         assertEquals(7, equipo2.getPuntos());
31     }
32 }
```

Pruebas unitarias JUnit5



The screenshot shows an IDE window for a project named 'LaLiga-Fantasy-Generator-1'. The left sidebar displays the project structure, with the 'Test' folder expanded, showing the 'testUnitariosProyecto' class. The main editor displays the code for 'testUnitariosProyecto.java'.

```
9 public class testUnitariosProyecto {
32
33
34 // Clase ficticia con el metodo a probar (puedes adaptarlo al nombre real)
35 static class Utilidades {
36     public String calcularTemporadaDesdeAnioActual() {
37         int anioActual = Year.now().getValue() % 100;
38         int siguienteAnio = (anioActual + 1) % 100;
39         return String.format("%02d/%02d", anioActual, siguienteAnio);
40     }
41 }
42
43 @Test
44 public void testCalcularTemporadaDesdeAnioActual() {
45     Utilidades util = new Utilidades();
46
47     int anioActual = Year.now().getValue() % 100;
48     int siguienteAnio = (anioActual + 1) % 100;
49     String esperado = String.format("%02d/%02d", anioActual, siguienteAnio);
50
51     assertEquals(esperado, util.calcularTemporadaDesdeAnioActual());
52 }
53
54 @Test
55 public void testActualizarDatos_Ganado() {
56     equipo1.actualizarDatos( golesAFavor: 3, golesEnContra: 1); // Gana 3-1
57
58     assertEquals( expected: 1, equipo1.getPJ());
59     assertEquals( expected: 3, equipo1.getGF());
60     assertEquals( expected: 1, equipo1.getGC());
61     assertEquals( expected: 1, equipo1.getPG());
62     assertEquals( expected: 0, equipo1.getPE());
63     assertEquals( expected: 0, equipo1.getPP());
64     assertEquals( expected: 3, equipo1.getPuntos());
65 }
```

Pruebas unitarias JUnit5



The screenshot shows an IDE with a project named 'LaLiga-Fantasy-Generator-1'. The left sidebar displays the project structure, including folders like '.idea', 'bin', 'src', 'Test', and 'out'. The 'Test' folder is expanded, showing the file 'testUnitariosProyecto.java'. The main editor displays the code for this file, which contains three unit tests for the 'testActualizarDatos' method.

```
9 public class testUnitariosProyecto {  
55     public void testActualizarDatos_Ganado() {  
59         assertEquals( expected: 3, equipo1.getGF());  
60         assertEquals( expected: 1, equipo1.getGC());  
61         assertEquals( expected: 1, equipo1.getPG());  
62         assertEquals( expected: 0, equipo1.getPE());  
63         assertEquals( expected: 0, equipo1.getPP());  
64         assertEquals( expected: 3, equipo1.getPuntos());  
65     }  
66  
67     @Test new *  
68     public void testActualizarDatos_Perdido() {  
69         equipo1.actualizarDatos( golesAFavor: 0, golesEnContra: 2); // Pierde 0-2  
70  
71         assertEquals( expected: 1, equipo1.getPJ());  
72         assertEquals( expected: 0, equipo1.getGF());  
73         assertEquals( expected: 2, equipo1.getGC());  
74         assertEquals( expected: 0, equipo1.getPG());  
75         assertEquals( expected: 0, equipo1.getPE());  
76         assertEquals( expected: 1, equipo1.getPP());  
77         assertEquals( expected: 0, equipo1.getPuntos());  
78     }  
79  
80     @Test new *  
81     public void testActualizarDatos_Empatado() {  
82         equipo2.actualizarDatos( golesAFavor: 2, golesEnContra: 2); // Empata 2-2  
83  
84         assertEquals( expected: 1, equipo2.getPJ());  
85         assertEquals( expected: 2, equipo2.getGF());  
86         assertEquals( expected: 2, equipo2.getGC());  
87         assertEquals( expected: 0, equipo2.getPG());  
88         assertEquals( expected: 1, equipo2.getPE());  
89         assertEquals( expected: 0, equipo2.getPP());  
90         assertEquals( expected: 1, equipo2.getPuntos());  
91     }  
92 }
```

Resultados pruebas unitarias JUnit5



The screenshot shows the 'Run' window of an IDE. The title bar indicates the test is 'testUnitariosProyecto'. The toolbar contains icons for running, debugging, and other test-related actions. The test results are displayed in a tree view on the left, showing a total of 33 ms for the test suite. The individual tests and their durations are listed below. On the right, a summary states 'Tests passed: 6 of 6 tests - 33 ms'. Below this, the command used to run the tests is shown: 'C:\Users\gabis\.jdk\openjdk-23.0.2\bin\java.exe ...'. The final status is 'Process finished with exit code 0'.

Test Name	Duration
testUnitariosProyecto	33 ms
testActualizarDatos_Ganado()	11 ms
testActualizarDatos_Perdido()	
testSumarPuntos()	
testCalcularTemporadaDesdeAnioActual()	22 ms
testGetNombre()	
testActualizarDatos_Empatado()	

✓ Tests passed: 6 of 6 tests - 33 ms

```
C:\Users\gabis\.jdk\openjdk-23.0.2\bin\java.exe ...
```

Process finished with exit code 0

Depuración y
Refactorización

Refactorización



NewButton



Antes

```
JButton btnNewButton = new JButton("Volver");
btnNewButton.setBounds(34, 29, 63, 23);
btnNewButton.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        pagina02Simulacion ventanaSimulacion = new pagina02Simulacion(conexion);
        ventanaSimulacion.setVisible(true);
        dispose();
    }
});
pagina02Simulacion_clasificacion.setLayout(null);
pagina02Simulacion_clasificacion.add(btnNewButton);
```

Después

```
JButton btnVolver = new JButton("Volver");
btnVolver.setBounds(34, 29, 63, 23);
btnVolver.addActionListener(new ActionListener() {
    public void actionPerformed(ActionEvent e) {
        pagina02Simulacion ventanaSimulacion = new pagina02Simulacion(conexion);
        ventanaSimulacion.setVisible(true);
        dispose();
    }
});
pagina02Simulacion_clasificacion.setLayout(null);
pagina02Simulacion_clasificacion.add(btnVolver);
```

JLabel lblInfoClasificación



Antes

```
JLabel lblInfoClasificacion = new JLabel(" 1. FC BARCELONA");  
lblInfoClasificacion.setBounds(34, 80, 461, 23);  
pagina02Simulacion_clasificacion.add(lblInfoClasificacion);
```

Después

```
JLabel lblClasificacion = new JLabel(" 1. FC BARCELONA");  
lblClasificacion.setBounds(34, 57, 461, 23);  
pagina02Simulacion_clasificacion.add(lblClasificacion);
```

CalcularGolesRealista



Antes

```
private int calcularGolesRealista(float valoracionEquipo, float valoracionRival, boolean esLocal) {  
    // Base mínima de goles para cualquier equipo  
    float base = 0.8f;  
  
    // Diferencia fuerte en valoración: multiplicamos por 2.0 para amplificar  
    float diferencia = valoracionEquipo - valoracionRival;  
    float mediaGoles = base + (diferencia * 2.0f);  
  
    // Bonus por ser local  
    if (esLocal) {  
        mediaGoles += 0.5f;  
    }  
  
    // Limitar media para evitar extremos negativos o muy altos  
    if (mediaGoles < 0.2f) mediaGoles = 0.2f;  
    if (mediaGoles > 5.0f) mediaGoles = 5.0f;  
  
    // Simulación de goles usando Poisson (aproximado)  
    int goles = 0;  
    double prob = Math.exp(-mediaGoles);  
    double acumulado = prob;  
    double randVal = random.nextDouble();  
}
```

Después

```
private int calcularGoles(float valoracionEquipo, float valoracionRival, boolean esLocal) {  
    // Base mínima de goles para cualquier equipo  
    float base = 0.8f;  
  
    // Diferencia fuerte en valoración: multiplicamos por 2.0 para amplificar  
    float diferencia = valoracionEquipo - valoracionRival;  
    float mediaGoles = base + (diferencia * 2.0f);  
  
    // Bonus por ser local  
    if (esLocal) {  
        mediaGoles += 0.5f;  
    }  
}
```


Antes

```
ImageIcon icono2 = new ImageIcon(pagina02Simulacion.class.getResource("/images/LaLiga_EA_Sports_2023_Vertical_Logo.png"));
Image imagen2 = icono2.getImage().getScaledInstance(lblFoto_02Simulacion.getWidth(), lblFoto_02Simulacion.getHeight(), Image.SCALE_SMOOTH);
ImageIcon iconoAjustado2 = new ImageIcon(imagen2);
lblFoto_02Simulacion.setIcon(iconoAjustado2);
```

Después

```
ImageIcon iconoLaLiga = new ImageIcon(pagina02Simulacion.class.getResource("/images/LaLiga_EA_Sports_2023_Vertical_Logo.png"));
Image imagen2 = iconoLaLiga.getImage().getScaledInstance(lblFoto_02Simulacion.getWidth(), lblFoto_02Simulacion.getHeight(), Image.SCALE_SMOOTH);
ImageIcon iconoAjustado2 = new ImageIcon(imagen2);
lblFoto_02Simulacion.setIcon(iconoAjustado2);
```

Documentación

JavaDoc



Ejemplo JavaDoc (main y conexión)



```
/**
 * Método main para iniciar la aplicación.
 * Intenta conectar con la base de datos y muestra la ventana principal.
 *
 * @param args Argumentos de línea de comando (no usados).
 */
public static void main(String[] args) {
    ConexionMySQL conexion = new ConexionMySQL("root", "1234", "laliga");
    try {
        conexion.conectar();
    } catch (SQLException e) {
        e.printStackTrace();
        JOptionPane.showMessageDialog(null, "Error al conectar con la base de datos: " + e.getMessage());
        System.exit(1);
    }

    EventQueue.invokeLater(() -> {
        try {
            pagina01Principal frame01Principal = new pagina01Principal(conexion);
            frame01Principal.setVisible(true);
        } catch (Exception e) {
            e.printStackTrace();
        }
    });
}
```

Ejemplo JavaDoc (método cargarEquiposDesdeBD)



```
/**
 * Carga la lista de equipos con sus valoraciones desde la base de datos.
 * Limpia la lista anterior y agrega los nuevos equipos cargados.
 */
public void cargarEquiposDesdeBD() {
    equiposLaLiga.clear();
    try {
        String consulta = "SELECT nombre, valoración FROM equipos";
        ResultSet rs = conexion.ejecutarSelect(consulta);
        while (rs.next()) {
            String nombre = rs.getString("nombre");
            float valoracion = rs.getFloat("valoración");
            Equipo equipo = new Equipo(nombre, valoracion, 0, 0, 0, 0, 0, 0, 0);
            equiposLaLiga.add(equipo);
        }
        rs.close();
    } catch (SQLException ex) {
        ex.printStackTrace();
        JOptionPane.showMessageDialog(null, "Error al cargar los equipos: " + ex.getMessage());
    }
}
```

Resultado generación JavaDoc



OVERVIEW

CLASS

TREE

INDEX

SEARCH

HELP

clasesProyecto > ClasificacionEquipo

Contents

Filter

Description

Constructor Summary

Method Summary

Constructor Details

ClasificacionEquipo(String, int, int, int, int, int, int, int)

Method Details

getNombre()

getPuntos()

getPJ()

getPG()

getPE()

getPP()

getGF()

getGC()

getDF()

Class ClasificacionEquipo

java.lang.Object[Ⓢ]
clasesProyecto.ClasificacionEquipo

public class **ClasificacionEquipo**
extends Object[Ⓢ]

Representa la clasificación de un equipo en la liga, con estadísticas básicas.

Constructor Summary

Constructors

Constructor	Description
ClasificacionEquipo (String [Ⓢ] nombre, int puntos, int PJ, int PG, int PE, int PP, int GF, int GC)	Constructor que inicializa la clasificación del equipo con los datos proporcionados.

Method Summary

All Methods

Instance Methods

Concrete Methods

Modifier and Type	Method	Description
int	getDF()	
int	getGC()	
int	getGF()	
String [Ⓢ]	getNombre()	
int	getPE()	
int	getPG()	
int	getPJ()	
int	getPP()	
int	getPuntos()	

Documentación

README.md





LaLiga-Fantasy-Simulator

 Simulador de Temporadas de Liga

Esta aplicación permite simular una temporada completa de LaLiga Española de fútbol, consultar estadísticas, y gestionar los datos de la temporada simulada a través de una interfaz gráfica sencilla e intuitiva.

 Configuración del Proyecto

 SGBD Elegido

Se ha utilizado MySQL Workbench 8.0.42 como sistema de gestión de base de datos (SGBD).

 Configuración del entorno

1. Driver JDBC de MySQL:

Descargar el archivo `mysql-connector-java-8.0.20.jar` desde el sitio oficial de MySQL.

Añadirlo al classpath del proyecto o configurarlo en el IDE (Eclipse, IntelliJ, NetBeans, etc.).



2. Configuración de conexión JDBC:

La conexión se realiza desde la clase ConexionBD.java.

Formato de la URL de conexión:

```
String url = "jdbc:mysql://localhost/<nombre_base_de_datos>?user=<usuario>&password=<contraseña>&us
```

Este formato incluye:

Protocolo JDBC: jdbc:mysql://

Host: localhost

Base de datos: /<BD>

Parámetros:

user y password: usuario y contraseña de la base de datos

useLegacyDatetimeCode=false: asegura compatibilidad con fechas y horas modernas

serverTimezone=<zona_horaria>: ajusta la zona horaria al sistema local

Usuario y contraseña deben coincidir con los definidos en la base de datos.



▶ Ejecución de la Aplicación

Requisitos Previos:

```
JDK 8 o superior.  
  
MySQL funcionando y accesible.  
  
Driver JDBC correctamente configurado.  
  
Importar el proyecto como proyecto Java.  
  
Añadir mysql-connector-java-8.0.20.jar a las dependencias.  
  
Ejecutar la clase pagina01Principal.java.
```

🚀 Funcionalidades Principales

1. Menú Principal

Al iniciar la aplicación, se presenta una ventana emergente con las siguientes opciones:

Simular Temporada

Ver Temporada

Borrar Datos Simulados

Salir



2. Simular Temporada

Al elegir Simular Temporada, el programa:

Recupera los equipos registrados en la base de datos.

Simula una temporada completa, generando todos los partidos de ida y vuelta.

Los resultados se determinan en base a:

Condición de local o visitante.



Valoración de cada equipo (de 0 a 5 estrellas).

Se registran automáticamente en la base de datos los siguientes datos para cada equipo:

Puntos



Partidos jugados

Partidos ganados / empatados / perdidos

Goles a favor / en contra

Diferencia de goles

Una vez finalizada la simulación, aparece una nueva ventana con tres opciones:

Consultar Resultados: Navegar jornada por jornada.

Consultar Clasificación: Ver la tabla de posiciones actualizada por jornada.

Volver al Menú Principal



3. Ver Temporada

Al elegir Ver Temporada, el programa enseña una ventana con tres opciones:

Consultar Resultados: Navegar jornada por jornada.

Consultar Clasificación: Ver la tabla de posiciones actualizada por jornada.

Volver al Menú Principal

En caso de que no haya ninguna temporada simulada, saltará una ventana de error.

4. Borrar Datos Simulados

Al seleccionar esta opción, el programa permite eliminar todos los registros de la temporada simulada.

En caso de intentar eliminar una temporada y no haya ninguna generada, se mostrará un mensaje de error.

5. Salir

Finaliza la ejecución del programa y cierra todas las ventanas activas.

Notas

Asegírate de tener bien generada la base de datos ejecutando el archivo LaLigaFantasyGenerator_DataBase.sql dentro de nuestro SGBD.

Asegúrate de tener cargados los equipos en la base de datos ejecutando el archivo laliga_equipos dentro de nuestro SGBD antes de simular una temporada.

Los datos generados se guardan automáticamente y se pueden consultar o eliminar posteriormente.

Autor

Desarrollado por Félix Caballero Peña y Gabriel Sánchez Heredia.